

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Organización Computacional

MANUAL Técnico

ARM64

Calculadora de Números Enteros

Nombre	Carnet
José Fernando Ramirez Ambrocio	202400195
Jackeline Stephany Rivera Argueta	202401685
Lesly Mariela Chojolán Rubio	201807024
Pedro Tomás Mejía Tol	201403515

Guatemala, septiembre de 2026

DESCRIPCIÓN GENERAL DEL FUNCIONAMIENTO



El programa es una calculadora de números enteros que corre completamente en consola, desarrollada en su totalidad en lenguaje ensamblador ARM64 (AArch64) puro, sin utilizar código en C ni ninguna lógica híbrida. La entrada y salida se realiza directamente mediante llamadas al sistema (syscalls) de Linux: **read** (número 63) para leer del teclado y **write** (número 64) para escribir en pantalla, además de **exit** (número 93) para finalizar el programa de forma segura.

El punto de entrada del programa es la etiqueta **_start**, que da paso a un ciclo principal llamado **menu_loop**. Este ciclo se repite indefinidamente: imprime el menú de opciones, lee la opción elegida por el usuario, la convierte de texto a entero, valida que esté dentro del rango permitido (1 a 7) y, según el valor, transfiere el control a la subrutina correspondiente mediante comparaciones (**cmp**) y saltos condicionales (**beq**). Cada subrutina de operación regresa al mismo ciclo (**b menu_loop**) al terminar, lo cual permite ejecutar varias operaciones sin que el programa finalice después de cada cálculo, tal como lo exige el enunciado.

1.1 ORGANIZACIÓN DE CÓDIGO FUENTE

1

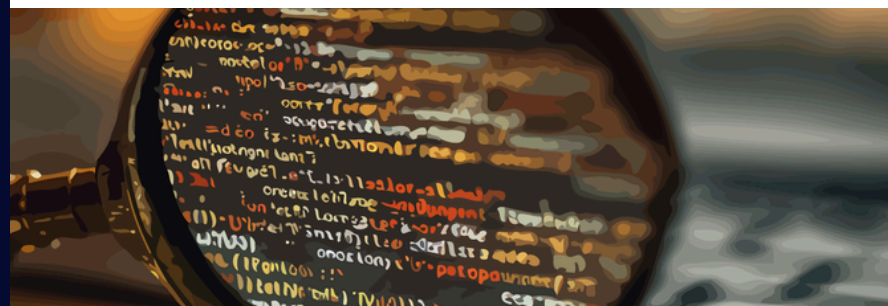
.data: contiene todos los mensajes de texto del programa (menú, solicitudes de números, mensajes de error y de resultado) como cadenas terminadas en nulo (**.asciz**), junto con su tamaño calculado en tiempo de ensamblado mediante **.equ**

2

.bss: reserva dos buffers sin inicializar de 16 bytes cada uno **buffer_entrada**, donde se guarda lo que el usuario escribe, y **buffer_salida**, usado para construir la representación en texto de los resultados numéricos.

3

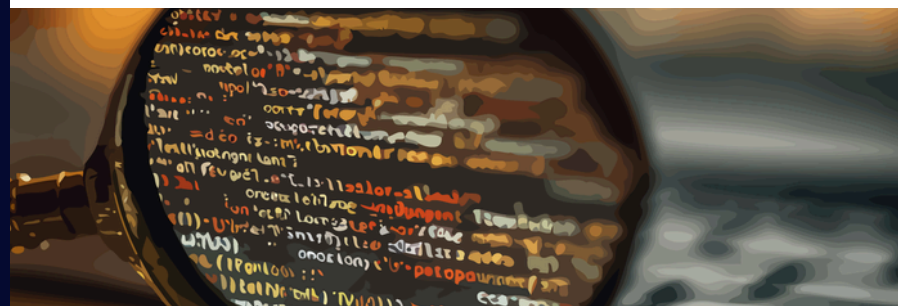
.text: contiene el código ejecutable el ciclo principal, las subrutinas auxiliares (**atoi**, **itoa**, **mensaje_error**) y las seis subrutinas de operaciones aritméticas.



1.2 SUBROUTINAS AUXILIARES

atoi recorre byte a byte el **buffer_entrada**, y por cada carácter válido ('0'-'9') multiplica el acumulador por 10 y le suma el nuevo dígito, hasta encontrar el salto de línea (código ASCII 10). Si encuentra cualquier carácter fuera del rango de dígitos (incluido el signo '-'), retorna **-1** como código de error, lo cual es aprovechado por todas las subrutinas para rechazar entradas no numéricas y, como efecto colateral, cualquier número negativo.

itoa hace el proceso inverso: toma el entero resultante en el registro **w0** y lo convierte a texto, escribiendo los dígitos desde el final del **buffer_salida** hacia atrás mediante divisiones sucesivas entre 10 (**udiv/msub** para obtener cociente y residuo). Si el número es negativo, antepone el signo '-'. Al terminar, deja el registro **x1** apuntando al primer carácter escrito y **x2** con la cantidad de bytes, listos para ser usados directamente en la siguiente llamada al sistema **write**.



Operaciones Implementadas



El programa implementa las seis operaciones solicitadas, cada una en su propia subrutina, siguiendo un patrón común: solicitar el/los operando(s), convertirlos con `atoi`, calcular el resultado, convertirlo de vuelta a texto con `itoa` e imprimirlo



Operación	Subrutina	Instrucción clave	Descripción
Suma	Suma	add	Suma directa de los dos operandos ingresados.
Resta	resta	sub	Resta directa entre los dos operandos ingresados.
Multiplicación	mult	mul	Multiplicación directa entre los dos operandos.
División entera	division	sdiv	División entera con signo entre dividendo y divisor.
Potencia	pot	mul (repetida en ciclo)	Multiplica la base por sí misma tantas veces como indique el exponente, mediante un ciclo iterativo (no usa instrucción de potencia).
Factorial	fact	mul (repetida en ciclo)	Multiplica de forma iterativa los enteros desde 2 hasta n, acumulando el resultado.



Validaciones realizadas



Restricciones	Dónde se valida	Mecanismo
Opción de menú fuera de rango (menor a 1 o mayor a 7)	menu_loop	cmp w0, #1 / blt error_ingreso y cmp w0, #7 / bgt error_ingreso, antes de saltar a cualquier subrutina.
Entrada no numérica en cualquier operando	atoi + cada subrutina	atoi retorna -1 si encuentra un carácter que no es dígito; cada subrutina hace cmp w0, #-1 / beq error_ingreso después de llamar a atoi.
División no debe permitir divisor igual a cero	division	cmp w20, #0 / beq division_cero: si el divisor es 0, se imprime un mensaje específico (msg_div_cero) y se regresa al menú sin ejecutar sdiv.
Potencia solo con exponentes enteros no negativos	pot	atoi rechaza el signo '-' (nunca convierte un negativo), por lo que un exponente negativo nunca llega a ejecutarse; el ciclo de multiplicación además usa el exponente como contador de repeticiones, nunca como valor con signo.
Factorial solo con números enteros no negativos	fact	Mismo mecanismo: atoi impide que un valor negativo se convierta correctamente, por lo que la entrada se rechaza antes de llegar al ciclo del factorial

Nota: dado que atoi no reconoce el signo '-', ningún operando negativo puede ingresarse en ninguna operación del programa. Esto cumple de forma natural las restricciones de potencia y factorial, aunque también implica que suma y resta trabajan únicamente con operandos de entrada positivos (el resultado sí puede ser negativo, por ejemplo en una resta).

4. Compilación y ejecución



El programa se ensambla y enlaza utilizando GNU Assembler y GNU Linker para AArch64. En un equipo x86_64 se ejecuta mediante emulación con QEMU:

```
aarch64-linux-gnu-as -o calculadora.o calculadora.s  
aarch64-linux-gnu-ld -o calculadora calculadora.o  
qemu-aarch64 -L /usr/aarch64-linux-gnu ./calculadora
```



Para depurar el programa y verificar el contenido de los registros durante la ejecución, se utilizó GDB con soporte multiarquitectura, ejecutando el binario junto con el modo de depuración remota de QEMU:

```
qemu-aarch64 -g 1234 -L /usr/aarch64-linux-gnu ./calculadora  
  
gdb-multiarch ./calculadora (gdb) target remote  
(gdb) target remote localhost:1234  
(gdb) break fact  
(gdb) continue  
(gdb) info registers w19 w21 w22
```



Evidencia de ejecución



A continuación se incluyen las capturas de pantalla que evidencian el funcionamiento del programa: el menú principal, la ejecución exitosa de cada una de las seis operaciones, el manejo de las validaciones (opción inválida, división entre cero y entrada no numérica) y una sesión de depuración con GDB.

5.1 Menú principal

```
jsriv@Stephy:~/ACE1_Practica1$ qemu-aarch64 ./calculadora
====Calculadora ARM64====
1. Suma
2. Resta
3. Multiplicacion
4. Division Entera
5. Potencia
6. Factorial
7. Salir
```

5.2 Ejecución de la suma

```
jsriv@Stephy:~/ACE1_Practica1$ qemu-aarch64 ./calculadora
====Calculadora ARM64====
1. Suma
2. Resta
3. Multiplicacion
4. Division Entera
5. Potencia
6. Factorial
7. Salir
1
Ingrese el primer numero:3
Ingrese el segundo numero:4
```

5.3 Ejecución de la resta

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
2  
Ingrese el primer numero:4  
Ingrese el segundo numero:3  
El resultado es:1
```



5.4 Ejecución de la multiplicación

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
3  
Ingrese el primer numero:8  
Ingrese el segundo numero:8  
El resultado es:64
```



5.5 Ejecución de la división entera (caso normal)

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
4  
Ingrese el primer numero:45  
Ingrese el segundo numero:4  
El resultado es:11
```



5.6 Validación: división entre cero

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
4  
Ingrese el primer numero:10  
Ingrese el segundo numero:0  
Error: no se puede dividir entre cero
```



5.7 Ejecución de la potencia

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
5  
Ingrese numero base:45  
Ingrese potencia:0  
El resultado es:1
```



5.8 Ejecución del factorial



```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
6  
Ingrese numero (n!):6  
El resultado es:720
```

5.9 Validación: opción de menú fuera de rango

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
8  
Opcion Invalida, intente de nuevo
```



5.10 Validación: entrada no numérica

```
====Calculadora ARM64====  
1. Suma  
2. Resta  
3. Multiplicacion  
4. Division Entera  
5. Potencia  
6. Factorial  
7. Salir  
hola  
Opcion Invalida, intente de nuevo
```



5.11 Sesión de depuración con GDB

```
Breakpoint 2, fact_loop () at calculadora.s:547
547      cmp w22, w19
1: /d $w19 = 5
2: /d $w21 = 1
3: /d $w22 = 2
(gdb) █
```

```
Breakpoint 2, fact_loop () at calculadora.s:547
547      cmp w22, w19
1: /d $w19 = 5
2: /d $w21 = 120
3: /d $w22 = 6
(gdb) █
```



```
Breakpoint 2, fact_loop () at calculadora.s:547
547      cmp w22, w19
1: /d $w19 = 5
2: /d $w21 = 120
3: /d $w22 = 6
(gdb) delete 2
(gdb) break fact_imprimir
Breakpoint 3 at 0x40061c: file calculadora.s, line 555.
(gdb) continue
Continuing.
```



```
Breakpoint 3, fact_imprimir () at calculadora.s:555
555      mov x8, #64
1: /d $w19 = 5
2: /d $w21 = 120
3: /d $w22 = 6
(gdb) info registers w21
w21      0x78      120
(gdb) █
```

5.11 Sesión de depuración con GDB

```
jsriv@Stephy:~/ACE1_Practica1$ qemu-aarch64 -g 1234 ./calculadora
====Calculadora ARM64====
1. Suma
2. Resta
3. Multiplicacion
4. Division Entera
5. Potencia
6. Factorial
7. Salir
6
Ingrese numero (n!):5
El resultado es:120
====Calculadora ARM64====
1. Suma
2. Resta
3. Multiplicacion
4. Division Entera
5. Potencia
6. Factorial
7. Salir
[]
```

```
jsriv@Stephy:~/ACE1_Practica1$ gdb-multiarch -q ./calculadora
Breakpoint 2, fact_loop () at calculadora.s:547
547      cmp w22, w19
1: /d $w19 = 5
2: /d $w21 = 24
3: /d $w22 = 5
(gdb) continue
Continuing.

Breakpoint 2, fact_loop () at calculadora.s:547
547      cmp w22, w19
1: /d $w19 = 5
2: /d $w21 = 120
3: /d $w22 = 6
(gdb) delete 2
(gdb) break fact_imprimir
Breakpoint 3 at 0x40061c: file calculadora.s, line 555.
(gdb) continue
Continuing.

Breakpoint 3, fact_imprimir () at calculadora.s:555
555      mov x8, #64
1: /d $w19 = 5
2: /d $w21 = 120
3: /d $w22 = 6
(gdb) info registers w21
w21      0x78      120
(gdb) continue
Continuing.
```



Conclusión



El desarrollo de esta práctica permitió aplicar de forma integrada los conceptos fundamentales de programación en ensamblador ARM64: manejo de registros, llamadas al sistema para entrada/salida por consola, comparaciones y saltos condicionales, ciclos iterativos y organización del código mediante subrutinas independientes. Implementar la potencia y el factorial mediante ciclos de multiplicación repetida, en lugar de instrucciones de más alto nivel, reforzó la comprensión de cómo operaciones matemáticas que en lenguajes de alto nivel se dan por sentadas deben construirse explícitamente a bajo nivel.